

LMS FULL-STACK Implementation Guide

Feature-by-Feature · Frontend First · Then Backend

Next.js 14 · Paystack · Flutterwave · Stripe · Prisma · Supabase

HOW TO USE THIS GUIDE

This guide is split into **7 phases** matching the recommended build order in your spec. Each phase contains:

- FRONTEND section — UI components, pages, and state to build first
- BACKEND section — API routes, database models, and server logic
- AI Agent Prompt — copy-paste prompt to use with Claude / Cursor / Copilot for that exact section

Complete the Frontend and Backend of each phase before moving to the next.

Phase Overview

Phase	Name	Key Deliverables	Est.
1	Auth & User System	Registration · Login · Roles · Profiles	1 week
2	Course Frontend	Listings · Course Page · Video Player · Search	2 weeks
3	Payment Integration	Paystack · Flutterwave · Stripe · Webhooks	1–2 weeks
4	Trainer Backend	Course Builder · Curriculum · Analytics	2–3 weeks
5	Admin Backend	User Mgmt · Approvals · Revenue · Payouts	2 weeks
6	Student Dashboard	My Courses · Progress · Notes · Certificates	1–2 weeks
7	Advanced Features	Gamification · Live Sessions · Coupons	2–3 weeks

PHASE 1 Auth & User System

Est: 1 week

FRONTEND IMPLEMENTATION

Pages & Components to Build

- `app/(auth)/register/page.tsx` — Multi-step registration form: Email/Password, then role selector (Student / Trainer)
- `app/(auth)/login/page.tsx` — Login form with email+password, Google OAuth button, LinkedIn OAuth button, and 'Forgot Password' link
- `app/(auth)/forgot-password/page.tsx` — Email input with reset-link confirmation message
- `app/(auth)/reset-password/page.tsx` — Token-validated new-password form with strength indicator
- `components/auth/AuthCard.tsx` — Reusable wrapper card (logo, title, form slot) used by all auth pages
- `components/auth/SocialLoginButtons.tsx` — Google and LinkedIn OAuth buttons using NextAuth `signIn()`
- `components/auth/RoleSelector.tsx` — Step 2 of registration: Student vs Trainer card selector
- `app/(dashboard)/profile/page.tsx` — Profile setup: avatar upload, name, bio, location, job title, skills, social links
- `components/ui/AvatarUpload.tsx` — Drag-and-drop avatar uploader, preview, crop, and upload to S3/R2

State & Auth Provider

- Wrap `app/layout.tsx` with `SessionProvider` from `next-auth/react`
- Create `lib/auth.ts` — NextAuth config with credentials, Google, and LinkedIn providers
- use `getSession()` hook to gate protected routes; redirect to `/login` if unauthenticated
- Role-based middleware in `middleware.ts`: `/trainer/*` requires `TRAINER` role, `/admin/*` requires `ADMIN`

Files to Create

Key Dependencies

AI AGENT PROMPT

You are building a Next.js 14 App Router LMS. Phase 1: Auth & User System.

Create the following frontend components and pages using TypeScript, Tailwind CSS, and shadcn/ui:

1. Registration page (`app/(auth)/register/page.tsx`) – multi-step:
 - Step 1: email + password + confirm password with Zod validation.
 - Step 2: role selector card (Student / Trainer).
2. Login page with email/password form + Google + LinkedIn OAuth buttons.
3. Forgot Password page and Reset Password page.
4. Profile setup page – avatar upload, name, bio, skills, social links.
5. `middleware.ts` – protect `/trainer/*` and `/admin/*` routes by role.

Use `react-hook-form` + `@hookform/resolvers/zod` for all forms.

Use `next-auth` `getSession()` for auth state. Show skeleton loaders.

BACKEND IMPLEMENTATION

Database Schema (Prisma)

Create `prisma/schema.prisma` with the following models:

- **User** — id, email, password (hashed), name, avatar, role (STUDENT | TRAINER | ADMIN), bio, location, jobTitle, skills[], linkedIn, twitter, emailVerified, twoFactorEnabled, createdAt
- **Account** — NextAuth OAuth account link (provider, providerAccountId, userId)
- **Session** — NextAuth session store
- **VerificationToken** — for email verification and password reset

API Routes to Build

- **POST `/api/auth/[...nextauth]`** — NextAuth handler (auto-generated from `lib/auth.ts` config)

- **POST /api/auth/register** — Validate input with Zod, hash password with bcryptjs, create User in DB, send welcome email via Resend
- **POST /api/auth/forgot-password** — Lookup email, generate signed reset token (JWT, 1hr expiry), email reset link
- **POST /api/auth/reset-password** — Verify token, validate new password, hash and update in DB, invalidate token
- **GET /api/user/me** — Return current user profile (protected by session)
- **PATCH /api/user/me** — Update profile fields; validate with Zod; handle avatar URL update
- **POST /api/user/avatar** — UploadThing endpoint for avatar; returns URL, stored to User.avatar

Security Checklist

- Passwords hashed with bcryptjs (cost factor 12) — never stored in plain text
- Reset tokens are short-lived JWTs signed with AUTH_SECRET — invalidated after single use
- Rate-limit /api/auth/register and /api/auth/forgot-password (max 5 req/min per IP) using upstash/ratelimit or middleware
- CSRF protection handled automatically by NextAuth for all OAuth flows
- Email verification: send link on register, block login until verified

AI AGENT PROMPT

You are building a Next.js 14 App Router LMS. Phase 1: Auth & User System.

Create the following backend components using Prisma + PostgreSQL + NextAuth:

1. prisma/schema.prisma — User, Account, Session, VerificationToken models.
User has role enum (STUDENT | TRAINER | ADMIN), all profile fields.
2. lib/auth.ts — NextAuth config: Credentials provider (bcrypt verify),
Google OAuth provider, LinkedIn OAuth provider. Include PrismaAdapter.
3. POST /api/auth/register — validate with Zod, hash with bcryptjs(12),
create user, send welcome email via Resend (React Email template).
4. POST /api/auth/forgot-password — JWT reset token (1hr), email link.
5. POST /api/auth/reset-password — verify token, update hashed password.
6. PATCH /api/user/me — update profile; UploadThing for avatar upload.

Add rate limiting to register + forgot-password endpoints.

PHASE 2 Course Frontend

Est: 2 weeks

■ FRONTEND IMPLEMENTATION

Pages & Components to Build

- **app/(public)/page.tsx** — Homepage: hero section, search bar, social proof counters, featured courses carousel, category tiles, trending section
- **app/(public)/courses/page.tsx** — Course discovery: filter sidebar (category, level, price, rating, language), search results grid, pagination
- **app/(public)/courses/[slug]/page.tsx** — Course detail: overview, instructor card, curriculum accordion, reviews, pricing CTA, sidebar
- **components/course/CourseCard.tsx** — Thumbnail, title, instructor avatar, star rating, review count, duration, price, badge (Bestseller/New)
- **components/course/CourseFilters.tsx** — Controlled filter sidebar with category checkboxes, level radio, price range slider, rating stars
- **components/course/CurriculumAccordion.tsx** — Expandable sections; free preview lessons have a Play button, locked lessons show lock icon
- **components/course/ReviewsSection.tsx** — Star breakdown bar chart, individual review cards with helpful/unhelpful vote buttons
- **components/course/InstructorCard.tsx** — Photo, name, bio, stats (rating, students, courses), link to full profile
- **components/course/VideoPlayer.tsx** — Cloudflare Stream or HLS player; playback speed, quality select, captions, PiP, progress save
- **components/search/SearchBar.tsx** — Debounced Algolia/Meilisearch input with instant suggestions dropdown
- **components/home/HeroSection.tsx** — Animated headline, search input, social proof counters (useCountUp), featured card strip
- **components/home/CategoryGrid.tsx** — Icon tiles for Tech, Business, Design, Marketing, HR, Finance, Personal Development, etc.

State Management

- URL-based filter state — use `nuqs` or `next/navigation` `searchParams` for filters (shareable URLs)

AI AGENT PROMPT

You are building a Next.js 14 App Router LMS. Phase 2: Course Frontend.

Build the following using TypeScript, Tailwind CSS, and shadcn/ui:

1. Homepage (`app/(public)/page.tsx`): hero with search bar + social proof counters (animate on scroll), featured courses carousel (`embla-carousel`), category icon grid, trending courses section.
 2. Course listing page with URL-based filters (`nuqs`): category, level, price, rating, language. Server-side filtering via Next.js `searchParams`.
 3. Course detail page: overview tab, instructor card, curriculum accordion (free preview lessons playable, locked lessons show lock icon), reviews with star breakdown, pricing CTA with countdown timer.
 4. VideoPlayer component using `@vidstack/react` – playback speed, quality, captions toggle, picture-in-picture, save progress to `localStorage`.
 5. SearchBar with Meilisearch instant suggestions (debounced, 300ms).
- All pages must be fully responsive. Use `Suspense` + `skeleton` loaders.

■ BACKEND IMPLEMENTATION

Database Schema (Prisma — additions)

- **Course** — id, slug, title, subtitle, description, thumbnailUrl, price, level, language, categoryId, trainerId, status (DRAFT|PENDING|LIVE|ARCHIVED), isFeatured, totalDuration, createdAt
- **Category** — id, name, slug, iconUrl, parentId (for sub-categories)
- **Section** — id, courseId, title, order
- **Lesson** — id, sectionId, title, type (VIDEO|ARTICLE|QUIZ), videoUrl, duration, isFreePreview, order
- **Review** — id, courseId, studentId, rating, body, helpfulCount, createdAt
- **Wishlist** — id, userId, courseId

API Routes to Build

- **GET /api/courses** — Paginated, filterable course list. Accept query params: q, category, level, price (free|paid), minRating, language, page, limit. Use Prisma where filters. Return courses with avg rating computed.
- **GET /api/courses/[slug]** — Full course detail: course + sections + lessons (with isFreePreview) + instructor + reviews (paginated) + stats
- **GET /api/courses/featured** — Admin-marked featured courses for homepage carousel
- **GET /api/courses/trending** — Sort by enrollment velocity (enrollments in last 7 days) and rating
- **GET /api/categories** — All top-level categories with course count
- **POST /api/wishlist** — Add/remove course from user's wishlist (toggle)
- **GET /api/wishlist** — Return current user's wishlist
- **GET /api/search/suggest** — Proxy to Meilisearch for instant suggestions

Search Indexing

AI AGENT PROMPT

You are building a Next.js 14 App Router LMS. Phase 2: Course Frontend Backend.

Add to prisma/schema.prisma: Course, Category, Section, Lesson, Review, Wishlist models with all fields from the spec.

Build the following API routes:

1. GET /api/courses — paginated, filterable (q, category, level, price, minRating, language). Validate query params with Zod. Include avgRating.
2. GET /api/courses/[slug] — full course detail with sections, lessons (hide videoUrl for locked lessons unless enrolled), reviews, instructor.
3. GET /api/courses/featured — isFeatured=true, limit 8, cached 1hr.
4. GET /api/courses/trending — rank by enrollments in last 7 days + rating.
5. POST/GET /api/wishlist — toggle and fetch wishlist (auth required).
6. GET /api/search/suggest — proxy to Meilisearch autocomplete.

Index all LIVE courses to Meilisearch on publish. Use next/cache tags.

PHASE 3 Payment Integration

Est: 1–2 weeks

■ FRONTEND IMPLEMENTATION

Pages & Components to Build

- **app/(public)/checkout/page.tsx** — Cart/order summary: course thumbnails, title, price, coupon field, total, gateway selector
- **components/checkout/GatewaySelector.tsx** — Three branded radio buttons: Paystack, Flutterwave, Stripe — each with logo and brief descriptor
- **components/checkout/CouponInput.tsx** — Real-time coupon validation input: calls `/api/coupons/validate` on change (debounced), shows discount or error
- **components/checkout/OrderSummary.tsx** — Course list, subtotal, discount line, taxes, grand total
- **app/(public)/checkout/success/page.tsx** — Confirmation: order summary, receipt number, 'Start Learning' CTA, download invoice link
- **app/(public)/checkout/failed/page.tsx** — Error message from gateway, reason, 'Try Again' and 'Different Method' buttons
- **components/checkout/PaystackButton.tsx** — `usePaystackPayment` hook, launches Paystack Popup, calls `/api/payment/verify` on callback
- **components/checkout/FlutterwaveButton.tsx** — `useFlutterwave` hook, launches Flutterwave Inline, calls `/api/payment/verify` on callback
- **components/checkout/StripeCheckoutButton.tsx** — Redirects to Stripe Checkout session URL (server-generated), handles `return_url`

AI AGENT PROMPT

You are building a Next.js 14 LMS. Phase 3: Checkout & Payment Frontend.

Build checkout UI with TypeScript, Tailwind, and shadcn/ui:

1. Checkout page (`/checkout`): cart with course thumbnails, `CouponInput` (debounced real-time validation → `/api/coupons/validate`), `OrderSummary` showing subtotal + discount + tax + total, `GatewaySelector` radio group.
 2. `PaystackButton`: use `react-paystack` `usePaystackPayment` hook. On success callback, POST to `/api/payment/verify` with reference.
 3. `FlutterwaveButton`: use `flutterwave-react-v3` `useFlutterwave` hook.
 4. `StripeCheckoutButton`: POST to `/api/payment/stripe/initiate`, redirect to returned `sessionUrl`.
 5. `/checkout/success` page: show receipt, 'Start Learning' button, invoice download link. Poll `/api/enrollment/status` until confirmed.
 6. `/checkout/failed` page: show error reason, retry options.
- Disable buttons during processing. Show spinner. Use `react-hot-toast`.

■ BACKEND IMPLEMENTATION

Database Schema (Prisma — additions)

- **Order** — `id`, `userId`, `courseId`[], `totalAmount`, `currency`, `gateway` (PAYSTACK|FLUTTERWAVE|STRIPE), `gatewayRef`, `status` (PENDING|SUCCESS|FAILED|REFUNDED), `couponId`, `invoiceUrl`, `createdAt`
- **Enrollment** — `id`, `userId`, `courseId`, `orderId`, `enrolledAt`
- **Coupon** — `id`, `code`, `discountType` (PERCENT|FIXED), `discountValue`, `expiresAt`, `maxUses`, `usedCount`, `courseId` (null = global)
- **Refund** — `id`, `orderId`, `reason`, `status` (PENDING|APPROVED|REJECTED), `requestedAt`, `processedAt`

API Routes to Build

- **POST /api/payment/paystack/initiate** — Validate cart, apply coupon, create pending Order, return Paystack reference and public key
- **POST /api/payment/flutterwave/initiate** — Same as above but return Flutterwave tx_ref and public key
- **POST /api/payment/stripe/initiate** — Create Stripe Checkout Session with course line items; return session URL
- **POST /api/webhooks/paystack** — Verify Paystack signature (HMAC-SHA512), handle charge.success → update Order to SUCCESS, create Enrollment, queue trainer revenue
- **POST /api/webhooks/flutterwave** — Verify Flutterwave secret hash, handle charge.completed → same flow
- **POST /api/webhooks/stripe** — Verify Stripe signature (stripe.webhooks.constructEvent), handle payment_intent.succeeded
- **POST /api/coupons/validate** — Check code exists, not expired, not exceeded maxUses, matches courseId. Return discount amount.
- **GET /api/orders/[orderId]/invoice** — Generate PDF invoice (pdfkit or reportlab via script), upload to S3, return URL
- **POST /api/refunds** — Create Refund record (PENDING). Email admin for review.

AI AGENT PROMPT

You are building a Next.js 14 LMS. Phase 3: Payment Backend.

Add to prisma/schema.prisma: Order, Enrollment, Coupon, Refund, TrainerEarning models.

Build these API routes:

1. POST /api/payment/paystack/initiate – validate cart, apply coupon, create PENDING Order, return {reference, publicKey}.
2. POST /api/payment/flutterwave/initiate – same, return {txRef, publicKey}.
3. POST /api/payment/stripe/initiate – Stripe Checkout Session, return URL.
4. POST /api/webhooks/paystack – verify HMAC-SHA512 signature using PAYSTACK_SECRET_KEY. On charge.success: Order→SUCCESS, create Enrollment, calculate TrainerEarning (amount*(1-commissionRate)).
5. POST /api/webhooks/flutterwave – verify secret hash. Same DB flow.
6. POST /api/webhooks/stripe – stripe.webhooks.constructEvent verify.
7. POST /api/coupons/validate – check expiry, maxUses, courseId match.
8. GET /api/orders/[id]/invoice – generate PDF with pdfkit, upload to S3.

All webhooks: use rawBody, disable Next.js body parsing, idempotent.

PHASE 4 Trainer Backend

Est: 2–3 weeks

■ FRONTEND IMPLEMENTATION

Pages & Components to Build

- **app/(trainer)/trainer/dashboard/page.tsx** — Overview: earnings summary, enrollment count, course list cards, recent reviews, pending tasks
- **app/(trainer)/trainer/courses/new/page.tsx** — New course wizard: Step 1 basic info, Step 2 curriculum, Step 3 pricing & publish
- **app/(trainer)/trainer/courses/[id]/edit/page.tsx** — Full course editor: all tabs (Info, Curriculum, Pricing, Preview, Publish)
- **components/trainer/CurriculumBuilder.tsx** — Drag-and-drop (dnd-kit) section and lesson reordering; add/rename/delete sections and lessons
- **components/trainer/LessonEditor.tsx** — Per-lesson editor: type selector (Video/Article/Quiz/Assignment), file uploader, rich text, free preview toggle
- **components/trainer/VideoUploader.tsx** — UploadThing drop zone for MP4/MOV, progress bar, thumbnail auto-extract, Cloudflare Stream upload
- **components/trainer/QuizBuilder.tsx** — Add questions (multiple choice, true/false, short answer), set pass score, toggle answer visibility
- **components/trainer/StudentTable.tsx** — Enrolled students: name, email, progress %, last active, grade; filter and search
- **components/trainer/AnnouncementEditor.tsx** — Rich text post to all enrolled students; sends email notification
- **app/(trainer)/trainer/analytics/page.tsx** — Enrollment trend chart (Recharts), completion rate, lesson drop-off chart, quiz score averages

AI AGENT PROMPT

You are building a Next.js 14 LMS. Phase 4: Trainer Backend Frontend.

Build the trainer dashboard UI using TypeScript, Tailwind, shadcn/ui:

1. Trainer dashboard: earnings card, enrollment count, course list with quick stats, recent reviews, onboarding checklist for new trainers.
2. Course builder wizard (multi-step): Step 1 basic info (title, category, level, tags), Step 2 CurriculumBuilder with @dnd-kit drag-and-drop sections + lessons, Step 3 pricing + free preview toggles.
3. LessonEditor: type selector (Video/Article/Quiz/Assignment), VideoUploader with UploadThing progress bar, @tiptap/react rich text for articles.
4. QuizBuilder: add/remove questions, set pass score, answer toggle.
5. StudentTable with @tanstack/react-table: progress, last active, grade.
6. Analytics page: Recharts enrollment trend, drop-off chart, rating chart.
7. Earnings page: monthly revenue chart, payout history table, payout settings form (bank account or PayPal).

All trainer pages must be under app/(trainer)/trainer/ and gate with role.

■ BACKEND IMPLEMENTATION

Database Schema (Prisma — additions)

- **Quiz** — id, lessonId, passScore, showAnswers
- **QuizQuestion** — id, quizId, question, type, options[], correctAnswer, order
- **Assignment** — id, lessonId, title, description, fileUrl (brief PDF)
- **AssignmentSubmission** — id, assignmentId, studentId, fileUrl, grade, feedback, submittedAt, gradedAt

- **Announcement** — id, courseId, trainerId, title, body, createdAt
- **LiveSession** — id, courseId, trainerId, title, scheduledAt, duration, meetingLink, recordingUrl
- **TrainerProfile** — id, userId, bio, expertise, portfolio, verifiedAt, commissionRate

API Routes to Build

- **POST /api/trainer/courses** — Create new course draft (trainerId from session)
- **PATCH /api/trainer/courses/[id]** — Update course info, pricing, status
- **POST /api/trainer/courses/[id]/sections** — Add section with title and order
- **PATCH /api/trainer/courses/[id]/sections/[sectionId]** — Rename or reorder section
- **POST /api/trainer/courses/[id]/lessons** — Add lesson (type, title, videoUrl, isFreePreview)
- **PATCH /api/trainer/courses/[id]/lessons/[lessonId]** — Update lesson or reorder
- **POST /api/trainer/courses/[id]/submit-review** — Change status DRAFT → PENDING (triggers admin notification)
- **POST /api/trainer/upload/video** — UploadThing endpoint, returns URL; also queue Cloudflare Stream transcode job
- **GET /api/trainer/students** — List enrolled students across all trainer's courses with progress
- **POST /api/trainer/announcements** — Create announcement, email all enrolled students via Resend batch
- **GET /api/trainer/analytics/[courseId]** — Enrollment by day, lesson completions, drop-off per lesson, avg quiz score

AI AGENT PROMPT

You are building a Next.js 14 LMS. Phase 4: Trainer Backend API.

Add to prisma/schema.prisma: Quiz, QuizQuestion, Assignment, AssignmentSubmission, Announcement, LiveSession, TrainerProfile.

Build these API routes (all require TRAINER role in session):

1. POST/PATCH /api/trainer/courses and /[id] — create and update course.
2. POST/PATCH /api/trainer/courses/[id]/sections — add/rename/reorder.
3. POST/PATCH /api/trainer/courses/[id]/lessons — add/update/reorder.
4. POST /api/trainer/courses/[id]/submit-review — status→PENDING, create AdminNotification, email admin.
5. POST /api/trainer/upload/video — UploadThing handler, return videoUrl.
6. POST /api/trainer/announcements — create, Resend batch email to all enrolled students (use Resend batch API, max 100/call).
7. GET /api/trainer/analytics/[courseId] — enrollments/day (last 30d), lesson completion counts, drop-off per lesson, avg quiz score.
8. GET /api/trainer/earnings — TrainerEarning grouped by month+course.
9. POST /api/trainer/live-sessions — create, schedule reminder emails.

PHASE 5 Admin Backend

Est: 2 weeks

■ FRONTEND IMPLEMENTATION

Pages & Components to Build

- **app/(admin)/admin/page.tsx** — Admin home: KPI cards (users, courses, revenue today/month/all-time), revenue line chart (Recharts), alerts feed
- **app/(admin)/admin/users/page.tsx** — Searchable/filterable DataTable of all users: role badge, status, join date, action menu (suspend, ban, promote)
- **app/(admin)/admin/users/[id]/page.tsx** — User detail: profile, enrolled courses, order history, activity log, role change select
- **app/(admin)/admin/courses/page.tsx** — All courses table: status badge (Draft/Pending/Live/Archived), trainer, enrollment, rating, action menu
- **app/(admin)/admin/courses/review/page.tsx** — Pending approval queue: course preview panel, approve/reject/request-edit buttons with reason input
- **app/(admin)/admin/trainers/page.tsx** — Trainer list: applications queue tab + active trainers tab. Approve/reject with reason.
- **app/(admin)/admin/finance/page.tsx** — Revenue overview, gateway breakdown (pie chart), transaction log DataTable, refund queue
- **app/(admin)/admin/payouts/page.tsx** — Pending payout requests table, batch payout button, payout history
- **app/(admin)/admin/settings/page.tsx** — Tabbed: Branding, Email Templates (rich text), Categories, Coupons, Notifications, Currency/Tax

AI AGENT PROMPT

You are building a Next.js 14 LMS. Phase 5: Admin Backend Frontend.

Build all admin UI pages under app/(admin)/admin/ using TypeScript, Tailwind CSS, shadcn/ui, Recharts, and @tanstack/react-table.

1. Admin home: 4 KPI stat cards + RevenueChart (toggle daily/weekly/monthly + gateway filter) + alerts notification feed with action buttons.
 2. Users DataTable: search, role filter, status filter. Row actions: Suspend, Ban, Promote to Trainer, View Profile. Show confirmation dialogs.
 3. Course review queue: two-column layout – list on left, CourseReviewPanel on right showing full course preview. Approve/Reject/Request Edit.
 4. Finance page: revenue overview cards, gateway pie chart, transaction DataTable with date range filter and CSV export, refund request queue.
 5. Payouts page: pending payout table (trainer, amount, method), select checkboxes for batch payout, confirm modal, payout history.
 6. Settings page: Branding tab (logo upload, color picker), Email Templates tab (@tiptap editor per template), Categories CRUD, Coupon manager.
- Gate all /admin/* routes – redirect to /login if role !== ADMIN.

■ BACKEND IMPLEMENTATION

Database Schema (Prisma — additions)

- **AdminNotification** — id, type (TRAINER_APP|COURSE_REVIEW|REFUND|PAYOUT_FAIL), refId, message, read, createdAt
- **PlatformSetting** — key (string), value (JSON), updatedAt
- **CourseReview** (admin review log) — id, courseId, adminId, action (APPROVED|REJECTED|EDIT_REQUESTED), note, createdAt

- **Payout** — id, trainerId, amount, currency, gateway, status (PENDING|PROCESSING|PAID|FAILED), reference, createdAt

API Routes to Build

- **GET /api/admin/stats** — Total users, active students, live courses, revenue totals (today/month/all), pending item counts
- **GET /api/admin/users** — Paginated user list with role filter, search, status filter
- **PATCH /api/admin/users/[id]** — Update role, suspend, ban, unsuspend. Email user on status change.
- **POST /api/admin/users/bulk-email** — Query segment from DB, batch send via Resend. Log to BulkEmailLog.
- **GET /api/admin/courses** — All courses with all statuses, pagination, filters
- **POST /api/admin/courses/[id]/approve** — Status → LIVE, index to Meilisearch, email trainer
- **POST /api/admin/courses/[id]/reject** — Status → DRAFT, email trainer with reason, create CourseReview log
- **PATCH /api/admin/courses/[id]** — Admin edit any course field
- **POST /api/admin/courses/[id]/feature** — Toggle isFeatured flag
- **POST /api/admin/trainers/[id]/approve** — Set role TRAINER, create TrainerProfile, email approval
- **GET /api/admin/finance** — Revenue totals, gateway breakdown, transaction log (paginated)

AI AGENT PROMPT

You are building a Next.js 14 LMS. Phase 5: Admin Backend API.

Add to prisma/schema.prisma: AdminNotification, PlatformSetting, CourseReview, Payout models.

Build these API routes (all require ADMIN role in session):

1. GET /api/admin/stats — aggregate counts + revenue totals using Prisma `_count` and `_sum` with `groupBy`.
2. GET/PATCH /api/admin/users/[id] — fetch user detail, update role/status.
On suspend/ban: update `User.status`, send templated email via Resend.
3. POST /api/admin/courses/[id]/approve — `Course.status→LIVE`, index to Meilisearch (`meilisearchClient.index('courses').addDocuments([...])`), email trainer with Resend.
4. POST /api/admin/courses/[id]/reject — `Course.status→DRAFT`, create CourseReview log, email trainer with rejection reason.
5. POST /api/admin/refunds/[id]/approve — lookup `Order.gateway`, call correct gateway refund API, update `Order.status→REFUNDED`.
6. POST /api/admin/payouts/process — loop selected earnings, call Paystack Transfer or Stripe Connect payout, create Payout record.
7. GET/PATCH /api/admin/settings — read/upsert PlatformSetting. Cache settings in memory with 5min TTL.

PHASE 6 Student Dashboard

Est: 1–2 weeks

■ FRONTEND IMPLEMENTATION

Pages & Components to Build

- **app/(student)/my-courses/page.tsx** — Grid of enrolled courses with progress bars and quick-resume buttons
- **app/(student)/learn/[courseSlug]/[lessonId]/page.tsx** — Full learning interface: video player + sidebar (curriculum, notes, Q&A; tabs)
- **components/learn/LessonSidebar.tsx** — Three-tab panel: Curriculum (lesson list with checkmarks), Notes, Q&A; Discussion
- **components/learn/NotesEditor.tsx** — Textarea that saves timestamped notes per lesson; list of all notes with timestamps; export to PDF button
- **components/learn/QASection.tsx** — Question input, question list sorted by upvotes, nested answers, upvote button, 'Mark as answered' for instructors
- **components/learn/LessonCompletion.tsx** — Checkbox to mark lesson complete; auto-advance to next lesson option; completion confetti on 100%
- **components/learn/ProgressRing.tsx** — SVG ring showing overall course % and section-level progress bars

AI AGENT PROMPT

You are building a Next.js 14 LMS. Phase 6: Student Learning Dashboard.

Build the student learning interface using TypeScript, Tailwind, shadcn/ui:

1. My Courses page: grid of enrolled course cards with progress bars, 'Continue' button linking to last watched lesson (from CourseProgress).
2. Learn page (/learn/[courseSlug]/[lessonId]):
 - Main area: video player (@vidstack/react) or article content.
 - Right sidebar (LessonSidebar): 3 tabs:
 - Tab 1 Curriculum: lesson list with checkmarks, click to navigate.
 - Tab 2 Notes: textarea + timestamp button, notes list, export PDF.
 - Tab 3 Q&A: question input, question list with upvotes + answers.
 - Bottom: LessonCompletion checkbox + 'Next Lesson' button.
 - Show ProgressRing (SVG) for overall course completion %.
3. Certificate page: styled certificate (student name, course, date, QR code linking to /certificates/verify/[code]), download as PDF.
4. Student dashboard: recent activity, achievement badges grid, calendar widget showing live sessions.

Save watch progress every 30s via PATCH /api/learn/watchtime.

■ BACKEND IMPLEMENTATION

Database Schema (Prisma — additions)

- **LessonProgress** — id, userId, lessonId, completed (bool), watchedSeconds, completedAt
- **CourseProgress** — id, userId, courseId, completionPercent, lastLessonId, completedAt
- **Note** — id, userId, lessonId, body, timestampSeconds, createdAt
- **Question** — id, userId, lessonId, body, upvotes, answered, createdAt
- **Answer** — id, questionId, userId, body, upvotes, isInstructor, createdAt
- **Certificate** — id, userId, courseId, verificationCode (unique UUID), issuedAt
- **Achievement** — id, name, description, iconUrl, condition (JSON rule)
- **UserAchievement** — id, userId, achievementId, earnedAt

API Routes to Build

- **POST /api/learn/progress** — Upsert LessonProgress (completed=true), recalculate CourseProgress.completionPercent, check achievement triggers
- **PATCH /api/learn/watchtime** — Update LessonProgress.watchedSeconds periodically (every 30s from client)
- **GET /api/learn/progress/[courseId]** — Return all LessonProgress for user in this course
- **POST /api/notes** — Create note with lessonId, body, timestampSeconds
- **GET /api/notes/[lessonId]** — Return user's notes for a lesson

AI AGENT PROMPT

You are building a Next.js 14 LMS. Phase 6: Student Learning Backend.

Add to prisma/schema.prisma: LessonProgress, CourseProgress, Note, Question, Answer, Certificate, Achievement, UserAchievement.

Build these API routes:

1. POST /api/learn/progress — mark lesson complete, recalculate CourseProgress.completionPercent = completed/total lessons * 100. If completionPercent reaches 100, trigger certificate generation and check achievement rules.
2. PATCH /api/learn/watchtime — upsert LessonProgress.watchedSeconds. Debounce: only update if diff > 30s. Protect against replay attacks.
3. POST/GET /api/notes, GET /api/notes/export/[courseId] — CRUD notes. Export: generate PDF with pdfkit listing all notes by lesson+timestamp, upload to S3, return signed URL.
4. POST /api/qa/questions + /[id]/answers — create Q&A. Mark isInstructor if session user is course trainer. POST /[id]/upvote — toggle upvote.
5. GET /api/certificates/[courseId] — verify completion=100%, upsert Certificate with UUID verificationCode, generate PDF cert, email student.
6. GET /api/certificates/verify/[code] — public, no auth required.
7. Achievement engine: on progress update, check all Achievement.condition rules (e.g. {type:'complete_courses', count:5}). Award if met.

PHASE 7 Advanced Features

Est: 2–3 weeks

■ FRONTEND IMPLEMENTATION

Pages & Components to Build

- **components/coupon/CouponManager.tsx** (Admin) — Create coupon form: code, type (% or fixed), value, expiry, max uses, course scope; list with usage stats
- **app/(trainer)/trainer/live-sessions/schedule/page.tsx** — Session scheduler: date/time picker, duration, Zoom/Meet link, linked course dropdown
- **components/learn/LiveSessionCard.tsx** — Upcoming session: title, date/time countdown, join button (active 10min before), recording link after
- **app/(student)/assignments/page.tsx** — Assignment list: due date, submission status, upload button, grade display
- **components/learn/AssignmentUploader.tsx** — File drop zone, upload to S3, submit handler, confirmation
- **app/(trainer)/trainer/assignments/page.tsx** — Submission review: table of submissions per assignment, download, grade input (0-100 or pass/fail), feedback textarea

AI AGENT PROMPT

You are building a Next.js 14 LMS. Phase 7: Advanced Features Frontend.

Build the following components using TypeScript, Tailwind, shadcn/ui:

1. Coupon Manager (admin): create form (code, type, value, expiry, scope) + table listing all coupons with usage bar (usedCount/maxUses).
2. Live Session scheduler (trainer): date/time picker, duration select, Zoom/Meet link input, course selector. Show scheduled sessions list.
3. LiveSessionCard (student): countdown timer to session, 'Join' button enabled 10min before (compare now vs scheduledAt). Link to recording after session ends.
4. Assignments (student): due date, status badge, AssignmentUploader (UploadThing drag-drop, progress, submit). Show grade+feedback after.
5. Assignment submissions (trainer): DataTable of submissions per assignment, download file, grade input + feedback submit.
6. Public profile page: user avatar, bio, courses, achievement badges.
7. Gamification: BadgeShelf grid (earned solid, locked greyed out with tooltip), Leaderboard showing top 20 with current user rank.

■ BACKEND IMPLEMENTATION

API Routes to Build

- **POST /api/admin/coupons** — Create Coupon. Validate uniqueness of code, logical expiry date, max uses ≥ 1
- **GET /api/admin/coupons** — List all coupons with usedCount, remaining uses, expiry status
- **DELETE /api/admin/coupons/[id]** — Soft-delete (set expiresAt to past)
- **POST /api/trainer/live-sessions** — Create LiveSession, schedule Resend 'sendAt' reminder email for 24h and 1h before
- **GET /api/live-sessions/upcoming** — Return upcoming sessions for enrolled courses of current user
- **PATCH /api/trainer/live-sessions/[id]/recording** — Add recordingUrl, auto-create new Lesson in course curriculum
- **POST /api/assignments/[id]/submit** — Create AssignmentSubmission with UploadThing file URL
- **PATCH /api/trainer/submissions/[id]/grade** — Set grade + feedback, update AssignmentSubmission, notify student via email
- **GET /api/profiles/[username]** — Public profile: user info, courses, achievements, reviews (no PII)
- **GET /api/leaderboard** — Top 20 students by (courses completed $\times 10$ + lessons completed). Cache 1hr.
- **POST /api/admin/settings/email-templates/[key]** — Save email template HTML, stored in PlatformSetting

- **GET /api/admin/settings/email-templates/[key]** — Return template for editor

AI AGENT PROMPT

You are building a Next.js 14 LMS. Phase 7: Advanced Features Backend.

Build the following API routes and cron jobs:

1. POST/GET/DELETE /api/admin/coupons — full CRUD. Validate code uniqueness.
GET returns usedCount and remaining uses per coupon.
2. POST /api/trainer/live-sessions — create session. Use Resend 'sendAt' to schedule reminder emails: 24h before and 1h before to all enrolled.
3. PATCH /api/trainer/live-sessions/[id]/recording — add recordingUrl, automatically create a new Lesson (type VIDEO, title='Session Recording') in the course's last section.
4. POST /api/assignments/[id]/submit — UploadThing upload + create AssignmentSubmission. Notify trainer via email.
5. PATCH /api/trainer/submissions/[id]/grade — set grade+feedback, send student notification email with grade details.
6. GET /api/leaderboard — aggregate UserAchievement count + LessonProgress completed count. Cache with next/cache revalidate 3600.
7. Cron (Vercel Cron or pg_cron): daily inactive-student flag job, weekly payout reminder emails. All cron handlers idempotent.

Required Environment Variables

Variable	Description
DATABASE_URL	PostgreSQL connection string (Supabase/PlanetScale)
AUTH_SECRET	NextAuth secret (random 32-char string)
GOOGLE_CLIENT_ID / SECRET	Google OAuth app credentials
LINKEDIN_CLIENT_ID / SECRET	LinkedIn OAuth app credentials
PAYSTACK_SECRET_KEY	Paystack secret key (sk_live_...)
PAYSTACK_PUBLIC_KEY	Paystack public key (pk_live_...)
FLUTTERWAVE_SECRET_KEY	Flutterwave secret key
FLUTTERWAVE_PUBLIC_KEY	Flutterwave public key
STRIPE_SECRET_KEY	Stripe secret key (sk_live_...)
STRIPE_PUBLISHABLE_KEY	Stripe publishable key
STRIPE_WEBHOOK_SECRET	Stripe webhook endpoint secret
UPLOADTHING_SECRET / APP_ID	UploadThing credentials
CLOUDFLARE_R2_ACCESS_KEY	Cloudflare R2 bucket credentials
CLOUDFLARE_STREAM_TOKEN	Cloudflare Stream API token
RESEND_API_KEY	Resend transactional email API key
MEILISEARCH_HOST / KEY	Meilisearch instance URL and master key
NEXT_PUBLIC_APP_URL	Public base URL (https://yourdomain.com)

Recommended Project Structure

```

lms/
├── app/
│   ├── (public)/ # Public-facing pages
│   ├── (auth)/ # Login, register, reset
│   ├── (student)/ # Protected student pages
│   ├── (trainer)/trainer/ # Protected trainer pages
│   ├── (admin)/admin/ # Protected admin pages
│   ├── api/ # All API routes
│   ├── auth/
│   ├── courses/
│   ├── payment/
│   ├── webhooks/
│   ├── trainer/
│   ├── admin/
│   └── learn/
├── components/
│   ├── auth/ course/ checkout/
│   ├── learn/ trainer/ admin/
│   ├── gamification/ profile/
│   └── ui/ (shadcn primitives)
├── lib/
│   ├── auth.ts # NextAuth config
│   ├── prisma.ts # Prisma client singleton
│   ├── email.ts # Resend helpers
│   ├── storage.ts # S3/R2 helpers
│   └── payments/ # Gateway helpers

```



```
■ ■■■ search.ts # Meilisearch client
■■■ prisma/
■ ■■■ schema.prisma
■■■ emails/ # React Email templates
■■■ cron/ # Scheduled jobs
■■■ middleware.ts
■■■ .env.local
```

General Tips for Working with AI Agents

Be explicit about the stack	Always start prompts with: 'Next.js 14 App Router, TypeScript, Tailwind CSS, shadcn/ui, Prisma, PostgreSQL.'
One feature per prompt	Don't ask for an entire phase at once. Use the per-section prompts provided in this guide.
Provide existing schema	Paste your current prisma/schema.prisma into the prompt when asking for new models so the agent uses correct relations.
Specify file paths	Always tell the agent exactly where to create files using the paths listed in each phase.
Ask for Zod validation	Add 'validate all inputs with Zod' to every backend prompt — agents often skip this.
Request error handling	Add 'include try/catch with proper HTTP status codes' — agents often only write happy-path code.
Ask for TypeScript types	Add 'use full TypeScript types, no any' to get properly typed responses.
Iterate on UI	For frontend prompts, first get the layout working, then ask separately to 'add loading states, error states, and empty states'.